

G02 · 빌딩블록 — associative memory · fast-weight · SSM · TTT

이 권은 Titans·Miras·Atlas·TNT·Nested Learning·Sleep 여섯 편이 **공통으로 딴고 선 네 개의 기초 부품**을 다룬다. 부품은 associative memory(읽으면서 계속 학습시키는 작은 표), fast-weight/linear attention(매 토큰 조금씩 고쳐 쓰는 누적 행렬), SSM/Mamba(새 양동이에게이트를 단 상태), TTT(test-time 에 진짜로 작은 모델을 학습하는 층) 다. 목표는 증명이 아니라 **감각**이다 — 각 부품이 토큰당 어떤 연산을 몇 번 하고, state가 몇 byte이며, KV cache와 무엇이 다른지를 시스템 모델링에바로 쓸 수 있게 손에 쥐는 것. 수식은 보여주되 전부 풀어서 설명하고, 유도는 하지 않는다.

1. Associative memory — 읽으면서 계속 학습시키는 작은표

1.1 이미 매일 서빙하는연산

여러분은 associative memory를 이 권에서 처음 배우는 게 아니다. 이미 운영하고 있다. decode 한 스텝에서 attention이 하는 일 — query로 cache에쌓인 key들을 대조해 연관된 value를 꺼내는 것 — 이 바로연상 (association)이다.

직관 associative memory는 “key를 넣으면 value가 나오는 표”다. 전화번호부(이름 → 번호) 처럼. 차이는 이 표를 **어떻게 저장하느냐**에있다.

저장 방식은 두 극단이 있다.

- **non-parametric (KV cache):** 모든 (key, value) 쌍을 그대로 따로보관한다. 정확하지만 표가 토큰 수 L 에 비례해 계속 자란다.
- **parametric (matrix memory):** 고정 크기 행렬 하나에 모든 쌍을**겹쳐 쓴다**. 크기는 고정이지만 어딘가에서 반드시 정보를 잃는다.

이 권의 주인공은 후자다. 그리고 여섯 논문이 하려는 일은 한 문장으로요약된다: **attention의 연상 품질에 다가가되, state는 고정 크기로유지하기**.

비유 KV cache가 “메모를 계속 새 종이에 적어 쌓는 것”이라면, matrix memory는 “화이트보드 한 장에 계속 덧쓰는 것”이다. 종이는 안 겹치지만 무한히쌓이고, 화이트 보드는 크기가 일정하지만 글씨가 서로 겹쳐 번진다.

1.2 겹쳐 쓰기와 crosstalk

가장 오래된 저장법은 각 쌍을 곱해서 만든 작은 행렬 (outer product)을 전부더하는 것이다. 토큰이 하나 올 때마다:

$$W_t = W_{t-1} + v_t k_t^\top$$

기호 풀이 W_t = 지금까지의 memory 상태 (행렬 하나). k_t = 이번 토큰의 key(주소). v_t = 이번 토큰의 value(저장할 값). $v_t k_t^\top$ = value 와 key 를 곱해 만든 rank-1 행렬 (작은 “한 장의 기록”). t = 토큰순번.

이 식은 “이번 쌍을 기존 화이트보드에 그냥 더한다” 는 뜻이다. 이것을 **Hebbian write** 라고 부른다. 두 가지 성질이 중요하다: 쓰기 전에 읽지 않고 (**blind write**), 과거를 다시 불필요가 없다 (현재 토큰만 있으면 됨 → 하드웨어 친화적).

문제는 읽을 때 드러난다. key k_j 로 읽으면 원하는 v_j 가 나오지만, 다른 key 들과 겹치는 만큼 다른 value 들이 섞여 들어온다. 이 오염이 **crosstalk** 이다.

직관 crosstalk = 화이트보드에서 서로 다른 글씨가 겹쳐 번진 정도. 두 key 가 얼마나 닮았는지 (내적) 가 곧 번짐의 양이다. key 들이 완전히 직교하면 (서로안 닮으면) 번짐은 0 이다.

주의 이 실패는 cache miss 와 다르다. cache miss 는 “특정 항목 하나가 없다” 는 국소 사건이지만, crosstalk 은 **모든 read 가 동시에 조금씩 오염되는** 전역 사건이다. “꽉 찼다” 는 명시적 경계가 없고 품질이 연속적으로 저하된다. 그래서 여섯 논문은 이 암묵적 저하를 “학습된 지우기 (gate)” 로 명시화하려 애쓴다.

d 차원 공간에는 서로 직교하는 방향이 d 개뿐이다. 그래서 이 표에 **정확히** 담을 수 있는 쌍은 대략 d 개까지만다. **parameter** 는 d^2 개인데 **저장 능력** 은 $O(d)$ — 이 간극이 이권 전체를 관통하는 병목이다.

1.3 capacity 를 늘리는 사술: Hopfield → softmax

저장 능력 (capacity) 을 늘리는 고전적 사술이 있다. 이 사술을 알아두면 뒤의 논문들이 왜 그런 모양의 feature map 을 쓰는지 예측할 수 있다.

- **Hopfield network(1982)**: 손상된 패턴에서 원본을 복원하는 memory. capacity 가 약 $0.14d$ — 즉 여전히 $O(d)$.
- **dense Hopfield(2016)**: 에너지 함수의 “차수” 를 올려 key 를 더 높은 차원으로 lift 하면 capacity 가 d^{n-1} 로 폭증한다.
- **exponential 극한 = softmax attention**: 차수를 무한대로 보내면 그 복원 규칙이 정확히 **transformer** 의 **softmax attention** 이 된다.

핵심 에너지 차수를 올린다 = key 를 feature space 로 lift 한다 = capacity 가 올라간다; 극한에서 softmax attention 에 도달한다. Atlas(G05) 가 쓰는 polynomial feature map ϕ_p 와 exponential feature map ϕ^* 가 이 사술의 두 고리다.

한계 capacity 는 공짜가 아니다. key 를 차수 p 로 lift 하면 저장 차원이 d^p 규모로 커지고, memory 행렬도 함께 커진다. 예: $d_k = 128$, $p = 2$ 면 lift 된 차원이 약 8 천, state 가 32 KiB → 약 2 MiB 로 65 배 커진다. **capacity** 를 사는 통화는 **state** 의 **byte** 수다.

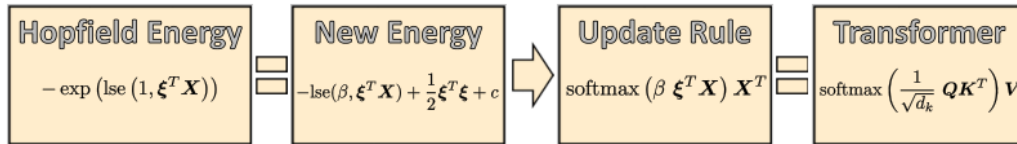


그림 1-1. Hopfield network의 에너지를 이진에서 연속 상태로 일반화하면 (가운데 log-sum-exp energy) 그 1-step 복원 규칙이 곧 transformer의 softmax attention이 된다 (오른쪽). “energy를 날카롭게 = key를 고차원으로 lift = capacity 상승, 극한에서 softmax”라는 사슬을 한줄로 보여 준다. 출처: Ramsauer et al., Hopfield Networks is All You Need (arXiv:2008.02217) Fig.1 — 원저자 그림.

1.4 delta rule — 틀린 만큼만 고쳐쓰기

Hebbian write의 결함은 blind라는 것이었다. 개선은 간단하다: 쓰기전에 먼저 읽어서, 지금 저장된 값과 목표의 차이 (오차) 만큼만 고쳐 쓴다.

$$W_t = W_{t-1} - \eta_t (W_{t-1} k_t - v_t) k_t^\top$$

기호 풀이 $W_{t-1} k_t$ = 이 key로 지금 읽어본 값 (예측). $W_{t-1} k_t - v_t$ = 예측과 목표의 차이 (오차 벡터). η_t = write 강도 (0에 가까우면 살짝만, 1이면 세게 고침). 나머지는 앞과 동일.

직관 이 식은 “이미 잘 저장돼 있으면 거의 안 건드리고, 틀린 만큼만 덮어쓴다”는 뜻이다. 같은 key가 두 번 오면 두 번째 write가 첫 번째를 덮어쓴다 — Hebbian 처럼 이중으로 쌓이지 않는다.

이것이 1960년에 나온 **delta rule**이다. 세 가지로 읽을 수 있고 세 이름 모두 뒤에서 계속 쓰인다.

1. **read-modify-write**: cache로 치면 append-only log가 아니라 주소를 찾아가는 in-place update다.
2. **1-step gradient descent**: $\ell(W) = \|W k_t - v_t\|^2$ 라는 손실을 한 걸음 내려간 것 그 자체다. “optimizer가 곧 memory write”라는 이 계열의 중심 명제가 여기서 처음 실체를 얻는다.
3. **부분 소거 후 재기록**: $\eta_t = 1$ 이면 이 key 방향의 옛 기억을 정확히 지우고 새 값을 쓴다. η_t 가 곧 write 강도를 조절하는 gate다.

주의 delta rule은 crosstalk을 교정하지만 **capacity 상한** $O(d)$ 자체는 못 올린다. 상한을 올리려면 memory를 깊게 (MLP) 만들거나 key를 lift해야 한다 (1.3절). “어떻게 쓸까”(write rule)와 “몇 개나 담을까”(capacity)는 독립된 두 축이다 — 이 구분이 뒤 논문들의 설계 공간을 가른다.

한 가지 냉정한 사실: 단 한 번의 online pass(sequence는 지나간 토큰을 다시 못 씀)에서 delta rule은 **최신 쌍을 정확히 쓰는 대신 옛 기억을 침범한다** (recency 편향). 이 구조

적 성질을 보완하려고 뒤 논문들이 momentum(G03), retention(G04), window 재최적화 (G05) 를 쌓는다.

시스템 모델링 관점 matrix memory 하나 (head 하나, $d_k = d_v = 128$, bf16) 를 모델링해 보자. - **state 크기**: $128 \times 128 \times 2 \text{ B} = 32 \text{ KiB}$ 고정. 문맥 길이와 무관. - **토큰당 read**: GEMV 한 번, $2d^2 \approx 33 \text{ KFLOPs}$, state 32 KiB 읽기. - **토큰당 write**: delta 면 예측 read(GEMV) + rank-1 교정 (outer product) → 대략 read 의 2 배 FLOPs + state 전체왕복 (RMW). - **exact 저장 한계**: head 당 약 $d_k = 128$ 쌍. byte 로는 KV cache 64 토큰과 맞먹지만, 문맥이 128 쌍을 넘는 순간부터 반드시 lossy. - **KV cache 대비**: state 를 1000 배 줄이는 대신, 저장 한계가 문맥길이와 무관한 head 당 상수로 고정된다. FLOP 과 byte 가 같은 $O(d^2)$ 차수라 arithmetic intensity 가 낮다 → 철저히 **bandwidth-bound**. - **주의**: W 가 요청마다 다른 per-sequence state 라, shared-weight 를 전제로 여러 요청을 한 GEMM 에 묶던 batching 감각이 깨진다.

2. Linear attention = fast-weight programming — 누적행렬로서의 KV cache

2.1 KV cache 를 고정 행렬로 압축하기

softmax attention 의 decode 는 매 스텝 cache 전체를 한 번 읽는다 — 그래서 문맥이 길 수록 bandwidth 를 먹는다. **linear attention** 은 similarity 에서 exp 를 빼서, cache 전체 대신 **누적 행렬** 하나만 유지하면 되게 만든다:

$$W_t = W_{t-1} + v_t k_t^\top, \quad y_t = W_t q_t$$

기호 풀이 W_t = 누적 행렬 ($d_v \times d_k$, head 당 하나). q_t = 이번 토큰의 query(read 주소). y_t = 이번 토큰의 출력. write 는 rank-1 outer product 를 더하는 것, read 는 GEMV 한 번.

낮익지 않은가? 이 **write** 는 1.2 절의 **Hebbian write** 와 문자 그대로 같은 식이다. 즉 linear attention 은 KV cache 를 고정 크기 행렬에 손실압축한 것이고, 겹쳐 쓰기의 대가는 그대로 crosstalk 이다. Titans(G03) 가 “linear attention 의 additive write 는 memory overflow 를 일으킨다” 고 비판하는 지점이 정확히 이것이다.

비유 KV cache append 는 “새 종이에 메모 추가”(간섭 없음). linear attention write 는 “같은 화이트보드에 덧쓰기”(간섭 = crosstalk). 이득의 정체는 연산이 빨라지는 게 아니라 **트래픽 총량에 상한이 생기는 것**이다— cache 가 안 자라니 decode 의 토큰당 byte 가 문맥과 무관한 상수가 된다.

2.2 fast weights 와 slow weights

이 권에서 가장 중요한 용어 구분이 여기 있다.

- **fast weights:** sequence 가 흐르는 동안 토큰마다 갱신되는 상태 —위의 W_t . 여러 분 세계의 “요청마다 존재하는 상태” (= KV cache 가 있던자리).
- **slow weights:** pretraining 이 끝나면 얼어붙는 보통의 parameter — projection W_K, W_V, W_Q 와 gate 를 만드는 작은 head 들. 여러분세계의 “모델” (= read-only checkpoint).

직관 fast weight programming(FWP) = 한 network(slow) 가 다른 network 의 weight(fast) 를 입력에 따라 써 넣는다는 1992 년의 아이디어. projection 이 slow net, outer-product write 가 프로그래밍 명령, read W_{tq_t} 가 프로그램 실행이다. 2021 년에 “linear attention 이 바로이 구도” 임이 밝혀지면서 이 계열의 역사적 기점이 됐다.

이 구분이 주는 실질적 이득: “이 gate 의 계수는 누가 정하나?” 라는 질문의 답이 항상 준비된다. write rule 자체는 inner loop(토큰마다) 의 사건이고, 그 계수를 만드는 함수 (예: $\eta_t = \eta(x_t; \Theta)$) 는 slow weights 의 일부로 outer loop(pretraining) 가 학습한다. 두 시간 척도가 항상 분리돼 있다.

2.3 계보: gate 와 write 를 한 손잡이씩 바꾸기

이 계열의 모든 변형은 같은 (state, update, cost) 객체에서 손잡이하나씩 바꾼 것이다. 두 축이 있다: retention(오래된 것 지우기) 과 write 교정 (틀린 것 고쳐쓰기).

retention 축 — 오래된 것 지우기. Hebbian write 는 지우는 수단이 없다 (eviction policy 없는 cache). 매 스텝 이전 상태를 조금 깎으면 된다:

$$W_t = \alpha_t W_{t-1} + v_t k_t^\top$$

기호 풀이 $\alpha_t \in [0, 1]$ = retention gate = 남기는 비율 ($\alpha_t = 1$ 이면 전부 유지, 작을수록 빨리 잊음). 역사적으로 “forget gate” 라 불렸다.

- **RetNet:** α 가 데이터와 무관한 고정 상수 (head 마다 다른 감쇠율).
- **GLA:** α_t 가 입력의 함수인 채널별 대각 gate (채널마다 남기는 비율이 다름).
- **Mamba-2:** α_t 가 입력의 함수인 스칼라 (3 장에서 다룸).

비유 gate = cache 항목의 TTL(time-to-live). $\alpha = 0.9$ 면 반감기가 약 6.6 토큰, $\alpha = 0.999$ 면 약 693 토큰. 채널별 gate 는 “같은 상태 행렬안에 두 자릿수 차이의 시간 척도를 채널 단위로 공존” 시킨다.

write 축 — 틀린 것 고쳐쓰기 (DeltaNet). gate 는 “전체를 흐리게” 할 뿐, “이 key 의 옛 값만 고쳐쓰기” 는 못 한다. 그건 1.4 절의 delta rule 이하는 일이고, 그것을 sequence 층으로 채택한 모델이 **DeltaNet** 이다.

Gated DeltaNet(GDN) 은 두 축의 합류다 — 전역지우기 (α_t) + 항목별 고쳐쓰기 (η_t) 를 한 update 에 실는다. 이 계열의 사실상 완성형이다.

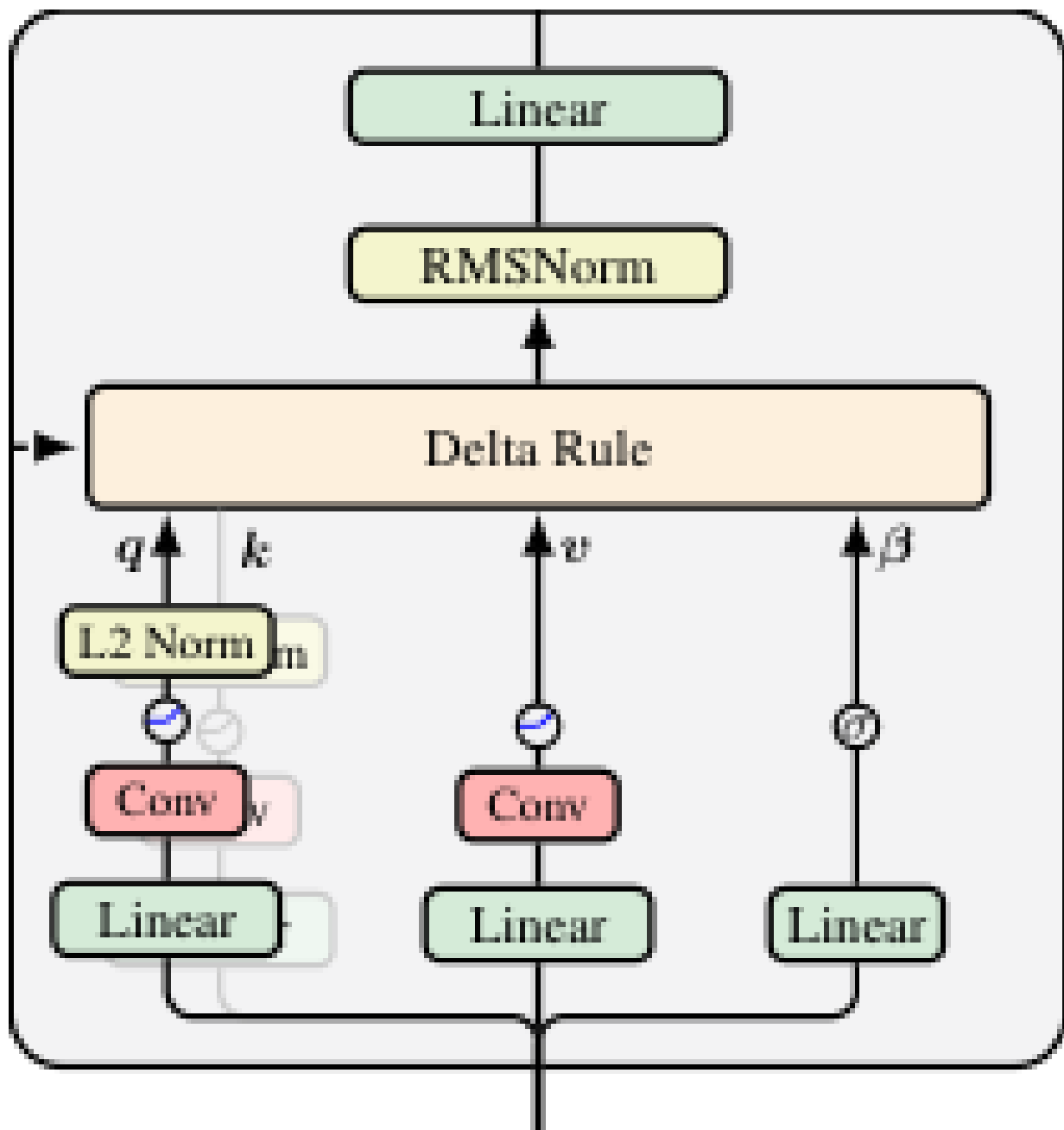


그림 2-1. DeltaNet 층의 구조. 매 토큰 (1) 현재 key로 memory를 먼저 읽어 예측을 만들고, (2) 목표 value와의 오차 (residual)만 outer product로 고쳐 쓰고, (3) query로 갱신된 memory를 읽는다. Hebbian의 “무조건 더하기”가 “틀린 만큼만 덮어쓰기”로 바뀐 것이 핵심. 출처: Yang et al., Parallelizing Linear Transformers with the Delta Rule (arXiv:2406.06484) Fig.2 — 원저자 그림.

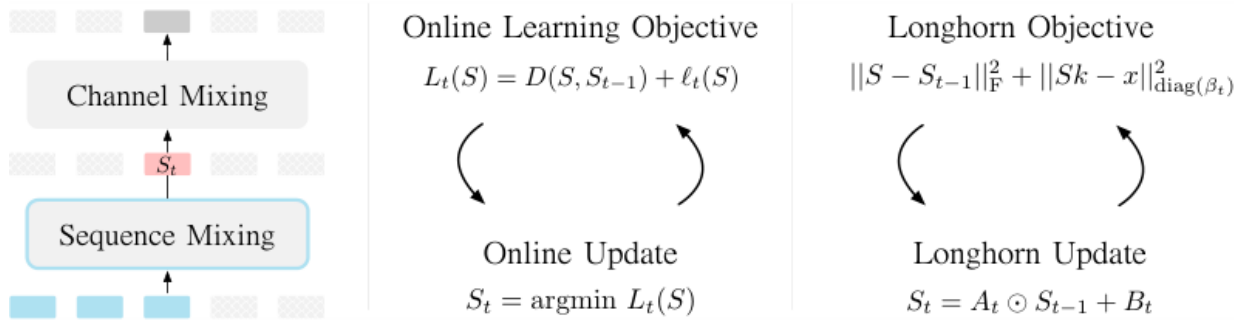


그림 2-2. sequence mixing 을 “history 를 state 로 압축하는 online learner” 로 보는 Longhorn 의 관점. 가운데는 일반형 목적함수 (이전 state 에가깝게 + 새 쌍을 잘 맞추게) 와 그 최적해 update, 오른쪽은 이를 proximal ℓ_2 로 특수화한 Longhorn 의 닫힌 해. 규칙을 짜맞추는 대신 문제를 정의하고 푼다. 출처: Liu et al., Longhorn (arXiv:2407.14207) Fig.2 — 원저자그림.

2.4 Longhorn — update rule 을 발명하지 말고 문제를 풀어라

지금까지는 update rule 을 설계했다. Longhorn 의 제안은 발상의 전환이다: 풀고 싶은 online 문제를 먼저 적고, 그 문제의 **정확한 해 (closed-form)** 를 update rule 로 삼는다.

문제는 “새 쌍을 잘 맞추되, 이전 상태에서 너무 멀어지지 말 것” 이고, 그해는 DeltaNet 과 형태가 같되 learning rate 가 하나 재정의된다:

$$\epsilon_t = \frac{\eta_t}{1 + \eta_t k_t^\top k_t}$$

기호 풀이 η_t = 원하는 write 강도 (게이트가 내놓는 값). ϵ_t = 실제로 적용되는 유효 강도. $k_t^\top k_t$ = key 의 크기 제곱. 분모가 항상 1 보다 크므로 ϵ_t 는 아무리 η_t 가 커도 폭주하지 않는다.

직관 보통의 gradient descent(explicit) 는 step size 를 너무 키우면 발산한다 — “절벽” 이 있다. Longhorn 은 그 절벽을 수식 안에서 없앤 **implicit gradient descent** 다. 어떤 $\eta_t > 0$ 을 넣어도 안 터지고, 극한에서 “이 key 에 v_t 를 정확히 저장하라” 는 hard write 로 수렴할 뿐이다. gate head 가 무슨 값을 내놓든 안정성이 보장된다.

시스템 모델링 관점 Longhorn 의 비용은 DeltaNet 과 사실상 동일하다 — ϵ_t 계산은 내적 하나 + 나눗셈 하나. state 도 그대로 $d_v \times d_k$ 행렬 하나. “공짜 안정성” 이 systems 요약이다. (단, 출하된 kernel 은 병렬화를 위해 비대각 transition 을 대각 근사판으로

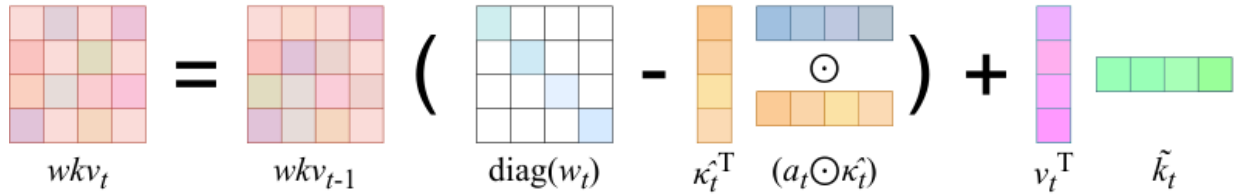


그림 2-3. RWKV-7의 state 갱신을 head 하나로 시각화 (실제는 head 당 64×64 , 그림은 4×4 축소). 대각 retention에서 “제거용 key”의 rank-1 항을 뺀 transition에 “기록용 key”의 write 항을 더한다. 지우는 key와 쓰는 key가 같은 key의 채널별 두 성형이라는 점이 한눈에 보인다. 출처: Peng et al., RWKV-7 “Goose” (arXiv:2503.14456) Fig.2 — 원저자 그림.

바뀌 쓴다 — 5장의 병렬화와 관련.)

2.5 RWKV-7 — gate를 채널별벡터로

RWKV-7은 이 계열의 현재 시점 최종 일반화다. GDN의 스칼라손잡이들을 전부 채널별 벡터로 승격하고, 지우는 key와 쓰는 key를 분리한다.

직관 GDN이 “cache line 단위 지우기 + line 단위 덮어쓰기”였다면, RWKV-7은 그 두 연산 모두에 **byte-enable** 신호를 단 것이다. 채널마다 다른 TTL, 채널마다 다른 write 강도, invalidate mask와 write mask를 따로. 제어자유도는 벡터로 늘었지만 저장소(행렬 하나)와 지배 비용(state RMW)은 그대로다.

이 계열 전체를 두 축의 격자로 입을 수 있다. 한 축은 retention의 해상도(상수 $\rightarrow$ 데이터 의존 스칼라 $\rightarrow$ 채널별 벡터), 다른 축은 write의 정밀도(Hebbian 더하기 $\rightarrow$ delta 덮어쓰기 $\rightarrow$ 지우기/쓰기 분리). RetNet은 retention 축 한 칸, DeltaNet은 write 축 한 칸, GDN은 둘 다 한 칸, RWKV-7은 두 축 모두 벡터 끝까지. Longhorn만 격자 밖에서 있다 — 좌표를 옮긴 게 아니라 explicit을 implicit으로 갈아끼워 안정성 자체를 재정의했으니까.

표 2-1 — 이 계열 카탈로그 (read는 모두 $y_t = W_t q_t$).

모델	write 요지	retention	한 줄 요약
linear attention	Hebbian 더하기	없음	누적 행렬, crosstalk 누적
RetNet	Hebbian 더하기	상수 decay	고정 TTL
GLA	Hebbian 더하기	학습된 채널별 대각	채널별 TTL
Mamba-2	Hebbian 더하기	학습된 스칼라	SSM 계보 (3장)
DeltaNet	delta 덮어쓰기	없음	틀린 것만 고쳐쓰기
Gated DeltaNet	delta 덮어쓰기	학습된 스칼라	지우기+고쳐쓰기완성형
Longhorn	implicit delta	없음	무조건 안정
RWKV-7	채널별 지우기/쓰기 분리	채널별 벡터	최종 일반화

시스템 모델링 관점 이 계열 전체의 decode 비용 (head 당, bf16, $d_k = d_v = d$): - **상주 state**: d^2 원소 고정 (RWKV-7 은 gate head 몇 개 추가, 소액). - **crossover**: state 가 KV cache 보다 작아지는 지점은 $L^* = d_k d_v / (d_k + d_v)$. $d = 128$ 이면 **64 토큰**. 단, GQA 로 KV head 를 8:1 로 줄이면 crossover 가 512 토큰으로 밀린다 — “linear attention 은 언제나 이득” 이 아니라 **자기 서빙 구성에 대입하는 산수**다. - **roofline**: decode 의 arithmetic intensity 가 약 1~1.5 FLOP/B → softmax attention 과 똑같이 깊은 bandwidth-bound. 이 계열의 승리는 강도 개선이 아니라 **트래픽절대량의 상한**이다. - **batching**: read/write 가 요청마다 다른 W_t 에 대한 GEMV/outer product 라, batch 축이 state 에 들어온다 → grouped-GEMM 류 kernel 필요. KV cache 관리자가 하던 일 (할당 · 상주 · 축출 · 격리) 을 fast-weight state 관리자가 물려받는다.

3. SSM / Mamba — 새는 양동이에 selective gate

3.1 새는 양동이와 게이트의 기원

state space model(SSM) 은 연속 시간 선형 시스템을 sequence 층으로 이식한 것이다. 고정 크기 buffer h 가 지금까지의 입력 스트림을 lossy 하게 압축해 들고 있다.

비유 SSM state = **새는 양동이 (leaky bucket)**. 매 스텝 옛 물은 조금 새어 나가고 (감쇠), 새 입력이 조금 부어진다. 얼마나 새고 얼마나 붓는지를 정하는 손잡이가 step size Δ 다.

연속 시스템을 토큰 단위로 계산하려면 discretization 이 필요한데, 그 결과는 6 장에서 본 linear recurrence 와 같은 구조다:

$$h_t = \bar{A} h_{t-1} + \bar{B} x_t, \quad \bar{A} = \exp(\Delta A)$$

기호 풀이 h_t = 고정 크기 상태 buffer. x_t = 이번 입력. Δ = step size (시간 해상도 손잡이). A = 감쇠를 정하는 행렬 (보통 음수 대각). $\bar{A} = \exp(\Delta A)$ = “지수 함수를 통과한 Δ ” = 남기는 비율. $\bar{B}$ = write 강도.

직관 Δ 가 작으면 $\bar{A} \rightarrow 1$ (거의 다 유지, 입력 거의 무시), Δ 가 크면 $\bar{A} \rightarrow 0$ (옛 것 다 버리고 현재 입력이 지배). 즉 Δ 는 “**얼마나 유지할지**” 와 “**얼마나 세게 쓸지**” 를 한 손잡이에 묶은 **gate** 다. 이 계열의 모든 gate — Mamba 의 Δ_t , Mamba-2·GLA 의 decay, Titans 의 weight decay, Miras 의 retention — 가 전부 이 한 줄의 후손이다.

3.2 Mamba — 게이트가 입력의 함수가되다

초기 SSM(S4) 은 **시불변 (LTI)** — Δ, A, B, C 가 토큰과 무관 — 이라 편리했다 (훈련을 convolution 으로 완전 병렬화). 대가는 표현력이다: 모든 토큰을 같은 비율로 감쇠시키니 “이 토큰은 기억, 저 토큰은 무시” 가 불가능하다.

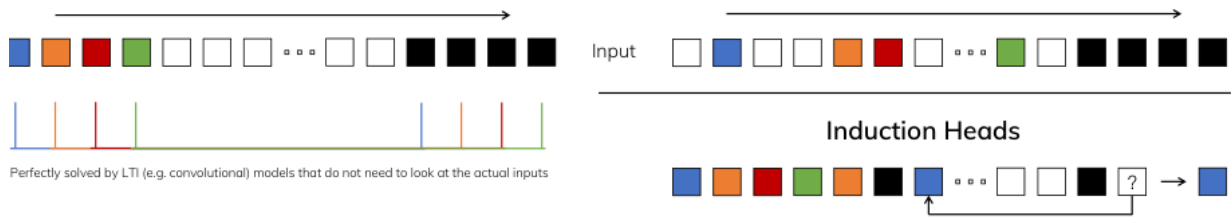


그림 3-1. Mamba가 selectivity를 요구하는 이유. 간격이 일정한 표준 copying(왼쪽)은 입력 내용을 볼 필요가 없어 시불변 모델이 완벽히 푼다. 그러나 간격이 무작위인 selective copying(오른쪽 위)과 문맥에 따라답을 회수하는 induction heads(오른쪽 아래)는 관련 토큰(색칠)을 무관토 큰(흰색)과 내용 기준으로 구별해야 하므로, 게이트를 입력의 함수로 만든 시변 모델이 필요하다. 출처: Gu & Dao, Mamba (arXiv:2312.00752) Fig.2 —원저자 그림.

Mamba의 해법은 **selectivity**다: Δ_t, B_t, C_t 를 입력 x_t 의 함수로 만든다. 그러면 모델이 토큰을 보고 “유지할까밀어낼까”를 결정한다.

주의 게이트가 시변이 되는 순간 convolution kernel이 사라진다 — 시불변을 깨면 FFT 병렬 모드가 소멸하고 순차 scan만 남는다. 그래서 Mamba는 scan을 SRAM 안에서 수행하는 IO-aware kernel(FlashAttention과 같은 부류)에 의존한다. 그런데 fusion은 트래픽만줄일 뿐, scan은 여전히 (1) 낮은 강도라 bandwidth-bound이고 (2) elementwise 연산이라 tensor core를 못 쓴다. 이 두 제약이 Mamba-2의 출발점이다.

3.3 Mamba-2와 SSD — 한 recurrence, 세 계산경로

Mamba-2의 첫 수는 **제약**이다: transition을 스칼라 곱으로 줄인다 ($\bar{A}_t = \alpha_t I$). 표현력을 조금 포기하는 대신, 그 update가 6장에서 이미 본 바로 그 식이 된다:

$$W_t = \alpha_t W_{t-1} + v_t k_t^\top, \quad y_t = W_t q_t$$

즉 SSM과 linear attention이 같은 식으로 합쳐진다. SSM 표기 B_t, C_t, x_t 가 각각 key k_t , query q_t , value v_t 에 대응한다. “SSM 이냐 linear attention 이냐”는 질문 자체가 소멸하고, “retention이 뭐고 write가 뭐냐”만 남는다.

SSD(state-space duality)는 이 recurrence가 계산하는 함수가 **decay mask**를 씌운 **attention**과 똑같다는 관계다.

같은 함수를 세 경로로 계산할 수 있다: - **recurrent (C=1)**: 토큰당 순차, decode 용. - **quadratic (C=L)**: $L \times L$ attention 처럼 완전 병렬, 짧은 prefill 용. - **chunkwise (1<C<L)**: 블록으로 잘라 안은 병렬, 경계만 순차 — 훈련 · 긴 prefill 용.

핵심 이 장의 세 경로는 전부 **exact**다. chunk 크기 C 는 결과를 바꾸지 않는 순수 성능 손잡이 — FlashAttention의 tile 크기와 정확히 같은 지위. “Mamba-2의 chunk는 **tiling(bit-exact)**이다.” 이 문장을 꼭 붙들어라. 5장에서 write가 gradient step으로 바뀌면 이 성질이 깨지고, 그때부터 C 는 함수 자체를 바꾸는 semantic 손잡이가 된다.

Structured State Space Model		Structured Masked Attention	
C	(contraction matrix)	Q	(queries)
B	(expansion matrix)	K	(keys)
X	(input sequence)	V	(values)
$A_{j:i}$	(state matrix)	L_{ji}	(mask)
N	(state expansion dim.)	N	(kernel feature dim.)
H (hidden states (8b)) $= L \cdot XB$ (linear mode)		SMA linear dual (15)	
SSM quadratic dual (16)		G (Gram matrix (13a)) $= Q \cdot K^T$ (quadratic mode)	

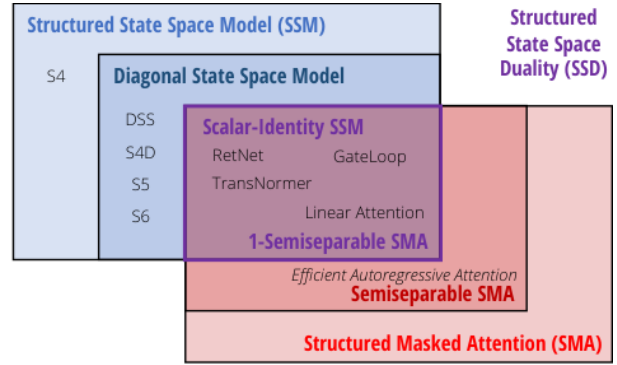


그림 3-2. state-space duality 의 지도. 왼쪽은 SSM 표기와 masked-attention 표기의 성분별 대응 ($C \leftrightarrow Q$ read 주소, $B \leftrightarrow K$ write 주소, $X \leftrightarrow V$ 값, 상태행렬 $A \leftrightarrow$ decay mask). 오른쪽은 SSM 집합과 structured attention 집합이 겹치는 영역이 곧 SSD 이며 RetNet·linear attention 이 그 교집합에놓임을 보인다. 같은 대상을 두 언어로 재서술한 것이다. 출처: Dao & Gu, Mamba-2 (arXiv:2405.21060) Fig.4 — 원저자 그림.

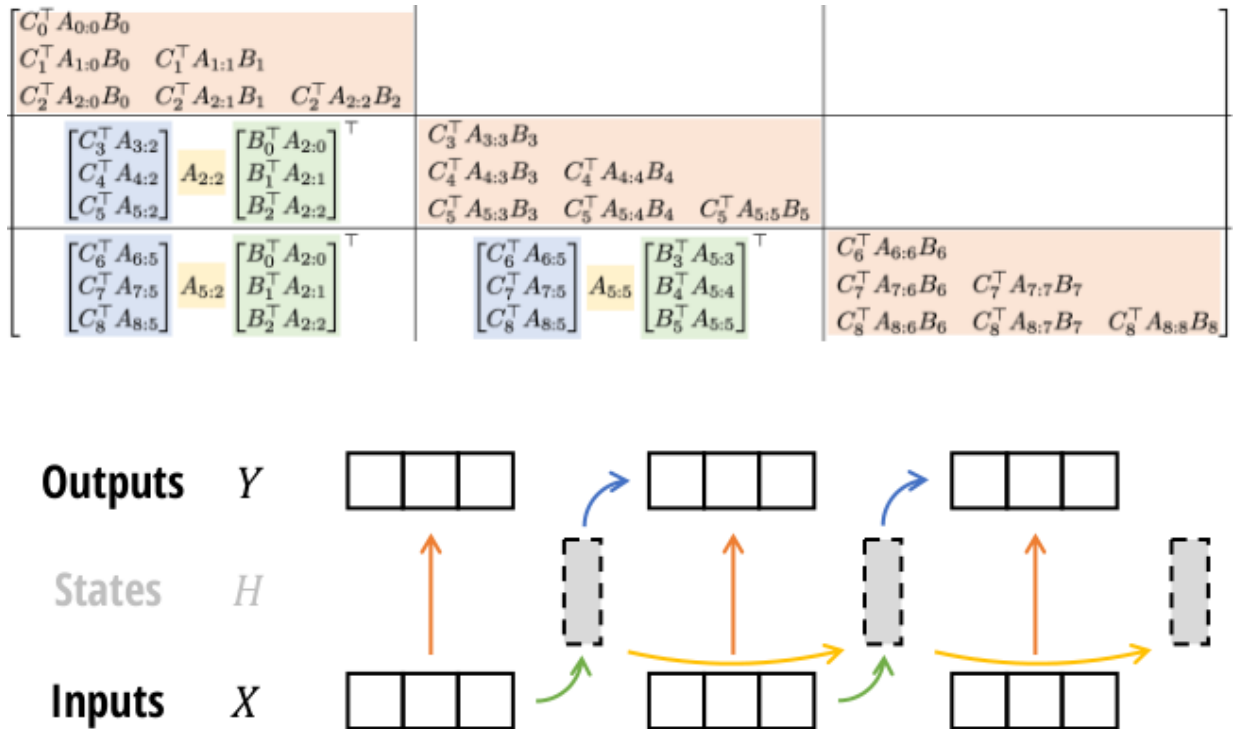


그림 3-3. SSD 의 chunk 분해. sequence mixing 행렬을 블록으로 나누면 대각 블록 (파랑) 은 chunk 내부의 attention 계산이 되고, off-diagonal 블록 (주황) 은 chunk 경계 상태를 통해 전달 되는 chunk 간기여가 된다. 아래 그림처럼 chunk 안은 병렬 (수직), 경계 상태만순차 (수평) 로 흐른다. 무거운 연산을 전부 GEMM 으로 바꿔 tensor core 를되찾는 것이 Mamba-2 속도의 핵심. 출처: Dao & Gu, Mamba-2 (arXiv:2405.21060) Fig.5 — 원저자 그림.

시스템 모델링 관점 Mamba/Mamba-2 decode: - **state**: 채널당 N 개 float, 폭 d 전 체로 $d \times N$ 행렬. 문맥이 1M 이어도 불변. - **추가 상태**: block 앞단의 짧은 causal conv 가 최근 몇 토큰 window 를 요구 → 서빙 엔진은 “SSM state + conv state” 묶 음을 요청마다 관리. paged KV cache 의할당 · 조각모음 문제는 사라지고, 고정 크 기 상태를 요청마다 하나씩 들고다니는 문제로 바뀐다. - **decode**: 두 세대 모두 per-token state RMW 로 같은 차수, bandwidth-bound. - **prefill/훈련**: Mamba-1 은 scan(bandwidth + op-mix 이중 제약), Mamba-2 는 chunkwise GEMM 으로 tensor core 회복 → fused scan 대비 2~8 배 빠름. - **감사습관**: “Mamba-2 가 빠 르다” 를 들으면 어느 **phase** 의 **throughput** 인지 먼저 물어라. decode 는 여전히 bandwidth 문제다.

4. TTT — test-time 에 진짜로 작은 모델을학습

4.1 inference 중 weight update 는 이미 검증된기법

여러분 세계의 가장 단단한 전제: “checkpoint 로드 후 weight 는 read-only.” TTT 계열 은 이 전제를 폐기한다. 하지만 이걸 2024 년의 발명이 아니라 vision 의 distribution-shift 문헌에서 십수 년 다듬어진 기법이다.

test-time adaptation 은 배포 중 만나는 입력에 맞춰 parameter 일부를 그 자리에서 갱신하는 기법이다. 핵심 통찰: **label 이 없어도 self-supervised loss(입력만으로 계산되 는 보조 손실) 로 학습 신호를 만들수 있다.** “training” 과 “inference” 의 구분은 시스템 관행이지 수학의제약이 아니다.

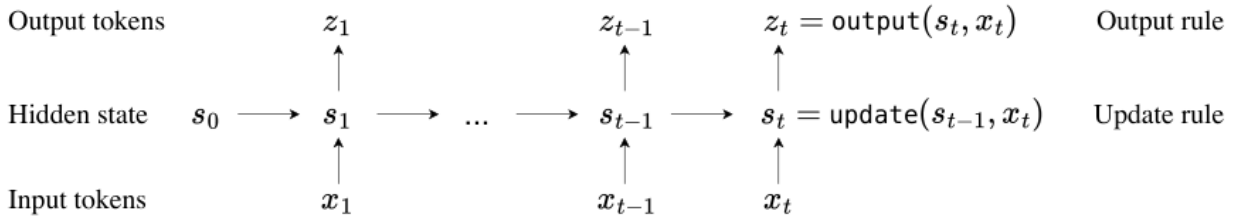
직관 vision 기법의 갱신 단위는 이미지 하나였다. 2024 년의 TTT layer 는 그단위를 토큰 하나 (= **decode** 스텝 하나) 로 좁히고, 갱신 대상을 backbone 이 아니라 층 안에 내장된 작은 **memory network** 로 한정했었다.

4.2 hidden state 가 곧 model 이다

6 장까지의 여정을 state 의 자료구조로 요약하면: vector state(Mamba) → matrix state(linear attention). 다음 단계는? **state 를 아예 작은 neural network 의 weight 전체로 승격시킨다.** 이것이 TTT 의 슬로건 — “hidden state is a model, and the update rule is a step of self-supervised learning.”

TTT layer 는 세 요소로 정의된다: 1. **inner objective**: 토큰의 한 view k_t 로 다른 view v_t 를 복원하도록 memory 를 학습 $-\ell(W) = \|\mathcal{M}(k_t; W) - v_t\|^2$. 2. **update**: 매 토큰 이손실에 1-step gradient descent. 3. **read**: 세 번째 view q_t 로갱신된 memory 를 읽음.

기호 풀이 $\mathcal{M}(\cdot; W)$ = 작은 memory network(그 weight 가 fast weights W). k_t, v_t, q_t = 입력 토큰을 세 가지로 투영한 것 (training view / label view / test view). η_t = 학습된 데이터 의존 inner learning rate(Mamba 의 게이트와 정확히 같은 역할). Θ = slow weights(projection 들과 게이트 head).



	Initial state	Update rule	Output rule	Cost
Naive RNN	$s_0 = \text{vector}()$	$s_t = \sigma(\theta_{ss}s_{t-1} + \theta_{sx}x_t)$	$z_t = \theta_{zs}s_t + \theta_{zx}x_t$	$O(1)$
Self-attention	$s_0 = \text{list}()$	$s_t = s_{t-1}.\text{append}(k_t, v_t)$	$z_t = V_t \text{softmax}(K_t^T q_t)$	$O(t)$
Naive TTT	$W_0 = f.\text{params}()$	$W_t = W_{t-1} - \eta \nabla \ell(W_{t-1}; x_t)$	$z_t = f(x_t; W_t)$	$O(1)$

그림 4-1. 모든 sequence modeling 층을 “hidden state + update rule” 의 한 틀로 보고, naive RNN·self-attention·TTT 를 세 요소 (초기 상태, update rule, output rule) 의 서로 다른 instantiation 으로 나란히 세운 그림. self-attention 은 상태가 (k,v) 리스트라 read 가 $O(t)$ 지만, TTT 는 상태가 고정 크기 W_t 라 read 가 $O(1)$ 이다. TTT 의 update rule 은 self-supervised loss 에 대한 gradient step. 출처: Sun et al., Learning to (Learn at Test Time) (arXiv:2407.04620) Fig.3 — 원저자 그림.

두 종류가 있다. **TTT-Linear** 는 $\mathcal{M}(k; W) = Wk$ (linear attention 과 같은 행렬 state). **TTT-MLP** 는 $\mathcal{M}$ 이 2-layer MLP (뒤 논문들이 표준화하는 deep memory 의 원형).

비유 TTT layer 의 fast weights = “압축 코덱이 달린, 쓰기 가능한 **KV cache**.” cache append 는 memcpy (FLOPs 0) 지만, TTT 의 write 는 read-modify-write 이고 그 “modify” 가 **gradient 계산**이다.

4.3 TTT-Linear 는 DeltaNet 이다

TTT-Linear 의 update 를 전개하면 새 발명이 아니라 재발견임이 드러난다. $\mathcal{M}(k; W) = Wk$ 를 손실에 넣고 미분하면 gradient 가 오차와 key 의 outer product — 즉 **delta rule** 과 같은 식이 나온다:

$$W_t = W_{t-1} (I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top$$

TTT-Linear = **retention gate** 없는 **DeltaNet**. 둘 다 1960 년 delta rule 의 sequence-layer 판본이다. 두 극한이 그림을 완성한다: chunk 하나로 전체를 잡고 초기 상태에서 gradient 를 평가하면 **linear attention** 이 되고, parametric 대신 non-parametric learner 를 넣으면 **softmax attention** 이 나온다. 즉 attention 도 linear attention 도 TTT 의 특수 사례다.

직관 Hebbian write 는 값을 무조건 더해 crosstalk 을 쌓지만, delta write 는 **residual** ($v_t - W_{t-1}k_t$) 만 쓴다. 이미 아는 내용이면 거의 안 쓰고 어긋난 만큼만 고친다. 이 “오차 기반 write” 가 Titans(G03) 가 **surprise** 라 부르게 될 것의 원형이다

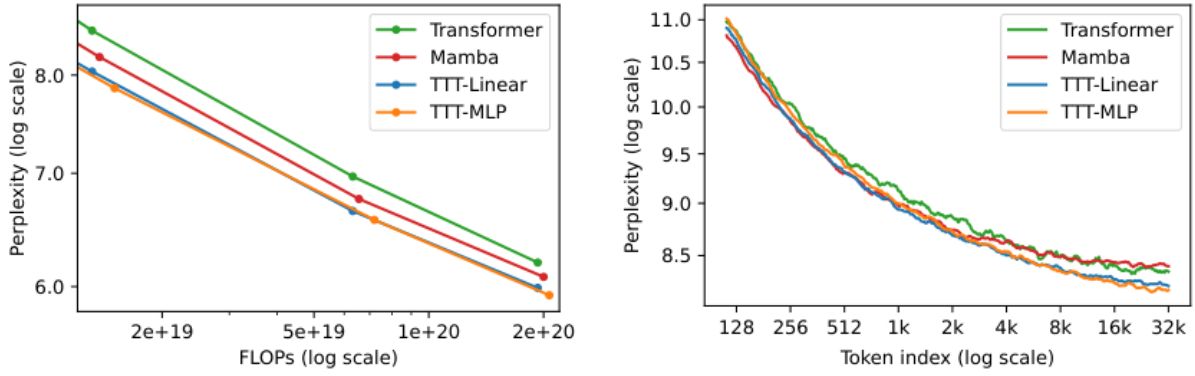


그림 4-2. TTT의 scaling 증거. 오른쪽: 토큰 index를 x축으로 한 perplexity — TTT-Linear·TTT-MLP·Transformer는 문맥이 길어질수록 계속내려가지만, Mamba는 뒤쪽에서 정체한다 (고정 크기 state의 capacity 한계, 1.2 절). 왼쪽: FLOPs 대비 perplexity로 봐도 이 이득이 compute-매칭에서유지됨. 출처: Sun et al., Learning to (Learn at Test Time) (arXiv:2407.04620) Fig.2 — 원저자 그림.

— surprise = 얼마나틀렸나.

4.4 이 층은 누가훈련하나

training 무경험 독자의 1 번 질문: “test time에 학습한다는데, 그 자체는누가 학습시키나?” 답은 **outer loop** = 보통의 **pretraining** 이다.

표 4-1 — TTT의 inner/outer 분업.

대상	소속	언제 움직이나	서빙 관점
W_t (memory 상태)	inner (fast)	매 토큰 · 매 요청	per-session state (KV cache 자리)
W_K, W_V, W_Q	outer (Θ)	pretraining에서만	checkpoint, read-only
η -head	outer (Θ)	pretraining에서만	checkpoint, read-only
W_{init} (초기 상태)	outer (Θ)	pretraining에서만	sequence 시작 시 W_0 로 복사

직관 inner update 식 전체가 미분 가능한 연산의 합성이라, outer의 gradient가 “풀어헤친 inner loop를 통과해” projection과 게이트 head까지 흘러간다. 핵심 명제: **inner optimizer의 hyperparameter(learning rate 같은)가, outer loop가 학습하는 데이터 의존 함수가 된다.** 폐기되는불변식은 정확히 한 칸이다 — W_t 만 움직이고 Θ 는 여전히 read-only.

한계 이 계열 전체 (TTT~Sleep) 의 실증 상한은 약 1.3B parameters / 100B tokens 수준이다. frontier-scale 서버에서 이긴다는 증거는 아직 없다. 이 권이 확립하는 건 “동작한다” 이지 “그 규모에서 이긴다” 가아니다.

시스템 모델링 관점 TTT decode 스텝 하나는 세 phase 다 — (1) write 의 loss/gradient 계산 (forward + **backward**), (2) write 적용, (3) 갱신된 상태로 read. 규칙은 update-then-read(먼저 쓰고 나서 읽음). - **TTT-Linear**($d_k = d_v = d$): read $\approx d^2$ MAC, write $\approx 2d^2$ MAC(예측 read + residual + outer product). **write** 가 **read** 와 같은 차수의 FLOPs. KV append(FLOPs 0) 에 익숙한사람에게 첫 구조 변화. - **TTT-MLP**(hidden $4d$): write 에 명시적 backward 가 들어옴 — write $\approx$ forward 의 4 배. “**backward $\approx 2 \times$ forward**” 라는 **훈련 비용 산식이 이제 decode latency 산식에 직접 들어온다**. - 세 가지 새 부담: (i) backward 가 decode 에 들어옴 (보통 gradient 를 손으로 유도해 fused kernel 로하드코딩), (ii) W_t 가 요청마다 달라 shared-weight batching 이 깨짐 $\rightarrow$ grouped-GEMM decode, (iii) per-session weight-state 의 sizing·checkpoint/restore·eviction. prefix snapshot 은 공유가 가능하지만 (deterministic recurrence), continuation 이 W 전체를수정하므로 fork 마다 copy-on-write 필요.

5. Chunkwise — 한 토큰씩 말고 블록단위로

5.1 훈련 속으로 들어온 decode loop

이 계열의 층은 정의상 per-token recurrence 다 — W_t 는 W_{t-1} 없이 못 만든다. decode 라면 당연하다. 문제는 **훈련**이다. 길이 L 전체를 per-token 으로 순차 실행하면 훈련이 통째로 decode loop 가된다.

비유 per-token 훈련 = **batch 1 짜리 decode** 를 32K 번 돌려서 **훈련 스텝하나를 만드는 것**. tensor core 는 놀고 wall-clock 은 무너진다. 실제로 작은 chunk 의 deep-memory 훈련은 peak FLOPs 의 5~10% 미만밖에 못쏜다.

chunkwise-parallel training 의 아이디어: sequence 를 크기 C 의블록으로 자르고, 블록 안은 병렬 (GEMM-rich) 로, 블록 사이 상태전달만 순차로 재조직한다.

5.2 일반 규칙: 경계에서 무엇을얼리는가

모든 chunkwise 기법은 update 를 두 부분으로 나눈다:

1. **state 에 linear 한 부분** (retention 곱 $\alpha_t W_{t-1}$, momentum EMA, Hebbian/delta write) — 이걸 **정확히** 병렬화된다. 누적곱과 가중합의 닫힌 형태, 또는 prefix-sum(associative scan) kernel 로.
2. **state 에 nonlinear 한 부분** (deep memory 의 gradient 가 MLP 를통과하는 항) — 여기엔 알려진 효율적 exact 병렬화가 없다. 그래서현실적인 수는 **얼리는 것**: chunk 안의 모든 gradient 를 chunk 시작 상태에서 평가한다.

직관 얼리기 (stale-snapshot) = “블록 안 모든 토큰이, 블록이 시작될때의 낡은 상태를 보고 gradient 를 계산한다.” 토큰 3 의 gradient 가 토큰 1·2 의 write 를 반영하지 못하고 블록 시작의 snapshot 을 본다. anchor 가 공유되니 블록 안 gradient 들이 서로 독립 → batch 로 병렬 계산 가능.

기호 풀이 C = chunk(블록) 크기. $\xi(t, C)$ = 토큰 t 가 속한 chunk 의 시작지점. g_t = anchor 에서 평가한 토큰별 gradient. inter-chunk handoff = 블록 사이로 넘기는 상태 하나.

블록 안은 $C \times C$ 삼각 구조의 GEMM(여러분이 FlashAttention tile 에서 매일 보는 QK^T 과 같은 shape) 이 되고, 블록 사이는 상태하나만 handoff 한다. 슬로건: “**chunk 안은 attention 처럼, chunk 경계는 RNN 처럼.**”

5.3 exact 인가 근사인가 — 판정 기준하나

핵심은 update 가 state 에 linear 한가다.

- **GLA / RetNet / Mamba-2 (스칼라 · 대각 linear)**: decay 를 elementwise 로 접으면 끝. **exact**.
- **DeltaNet (비대각 linear)**: Householder 곱은 elementwise 로 안접하지만, 수치 선형대수의 WY representation 으로 “교정된 value 들 사이의삼각 선형계” 를 풀면 된다. **여전히 exact**.
- **TTT / Titans (nonlinear, anchor 얼리기)**: 얼린 결과가 함수정의에 들어간다. 근사.

핵심 (명제). **chunk 크기 C 는 두 지위 중 하나다.** exact family(GLA·DeltaNet·Mamba-2) 에서 C 는 결과를 안 바꾸는 순수성능 손잡이 (tile 과 동급). anchor family(TTT·Titans·Atlas 의 deep memory) 에서 C 는 계산되는 함수 자체를 바꾸는 **semantic hyperparameter** 다 — 같은 slow weights 라도 C 가 다르면 다른 sequence-to-sequence 함수가 실행된다. $C = 1$ 이면 순수 online, $C = L$ 이면 linear attention 으로 퇴화, 그 사이는 전부 다른 층.

주의 이 구분이 실무 사고로 이어진다. semantic family 에서 train 과 serve 의 C 가 다르면 **훈련되지 않은 함수를 실행하는** 것이다. 실제로 $C = 64$ 로 훈련한 550M Titans 를 다른 chunk 로 돌리면 perplexity 가 $C = 64$ 에서 13.78 로 최적, $C = 8$ 에서 36.45, $C = 512$ 에서 22.4 로 무너진다. “작은 chunk = 신선한 gradient = 항상 좋음”이라는 직관이 틀린다 — 모델은 훈련된 해상도에 과적응한다. 이 mismatch 를 치유하는 게 TNT(G06) 의 주제다. 그리고 이 얼리기 근사의 오차에 대한 형식적 **bound** 는 여섯 논문 어디에도 없다 — 이 계열 최대의 미정량 리스크다.

5.4 roofline 위의 C

chunkwise 의 arithmetic intensity 를 C 의 함수로 쓰면 놀랄 만큼 깨끗한식이 나온다: 작은 chunk 에서 $AI(C) \approx C$. roofline 의 x 축 좌표를 chunk 크기 하나가 직접 쥐고 있는 셈이다.

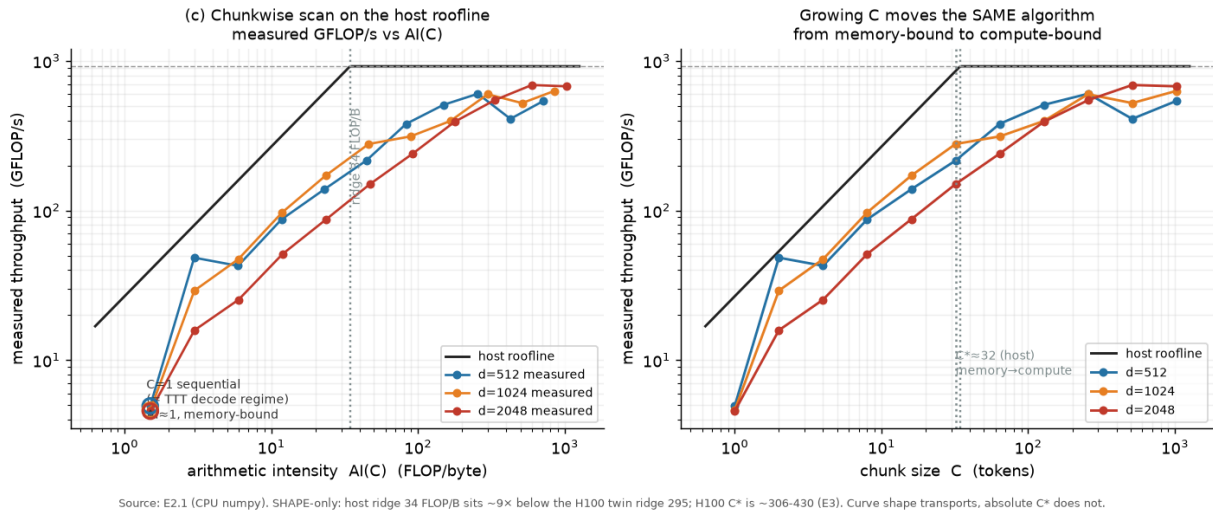


그림 5-1. chunkwise scan을 host roofline 위에 올린 자체 실험. 왼쪽: 측정된 GFLOP/s vs AI(C) — 작은 C(decode 영역)는 지붕의 낮은 처마 밑 (bandwidth-bound), C를 키우면 같은 알고리즘이 오른쪽 (compute-bound)로 이동한다. 오른쪽: C를 키우는 것이 곧 memory-bound $\rightarrow$ compute-bound 전환이라는 요지. 출처: 본서 자체 실험 (E2.1, host roofline). 비율·crossover 위치가 load-bearing 주장이며 절대 GB/s 값은 호스트 의존.

직관 품질이 원하는 $C(8\sim 64)$ 에서 kernel은 ridge의 3~20% 지점 (깊은 bandwidth-bound)에 앉고, MFU가 원하는 C 는 300 이상이다. **품질 최적 C 와 MFU 최적 C 는 같은 축의 반대편 끝에 있다.** exact family라면 이 긴장은 “훈련이 좀 느리다”로 끝나지만, semantic family에서는 C 가 품질 축이기도 하므로 **절충 불가능한 이율배반**이 된다 — 이것이 TNT(G06)가 존재하는 이유의 전부다.

시스템 모델링 관점 chunkwise를 비용으로 모델링할 때: - **판정 먼저:** update가 state-linear면 exact(C = 성능 knob), nonlinear면 anchor 근사(C = 함수 knob). 논문에서 “chunk 병렬화된다”를 보면 이 두 지위 중 어느쪽인지부터 물어라. - **비용:** chunk 당 intra $O(C^2d)$ + state 상호작용 $O(Cd^2)$. $C = 1$ 이면 순수 recurrence, $C = L$ 이면 순수 attention. - **deep memory 이중 부담:** batched forward/backward의 batch 축이 C 라 작은 C 에서 GEMM이 말라 tensor core 안 참, 그리고 momentum·gradient가 parameter-shape($\approx 8d^2$ 원소)라 scan 트래픽이 $O(C \cdot P_f)$ 로 커짐 $\rightarrow$ memory-bound. - **TNT의 처방 (G06 예고):** 큰 chunk global memory + 작은 chunk local memory 계층화, local 상태를 주기적으로 W_{init} 으로 **reset**해 블록들을 독립시켜 device에 분산 (context parallelism), train/serve의 C 를 분리하는 two-stage 훈련. semantic hyperparameter가 강요하던 단일 절충값을 두 개의 손잡이로 쪼갬다.

6. 시스템 모델링 관점에서 한장으로

네 부품을 여러분의 단위계 (byte, FLOP, bandwidth)로 결산한다.

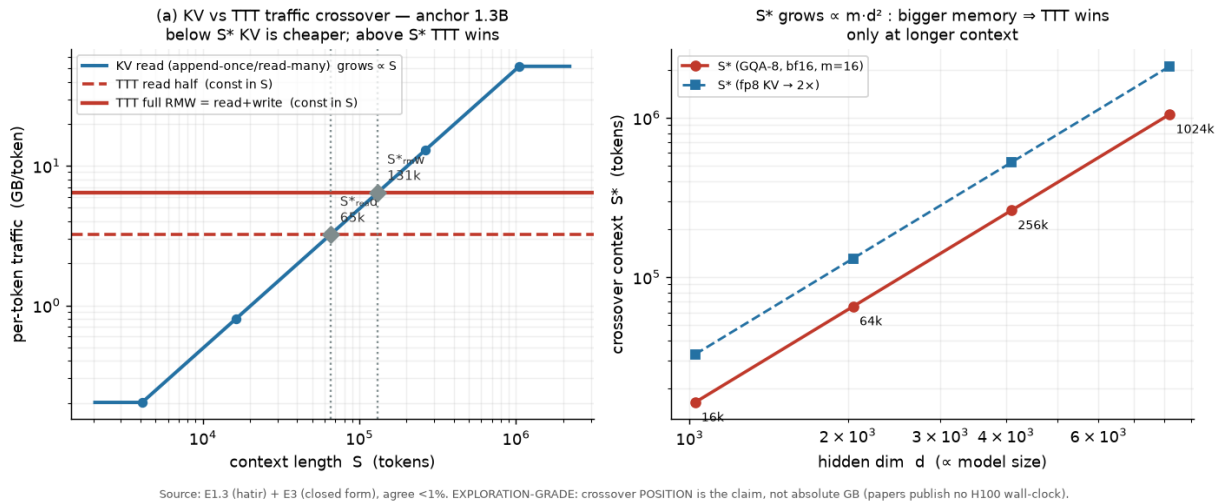


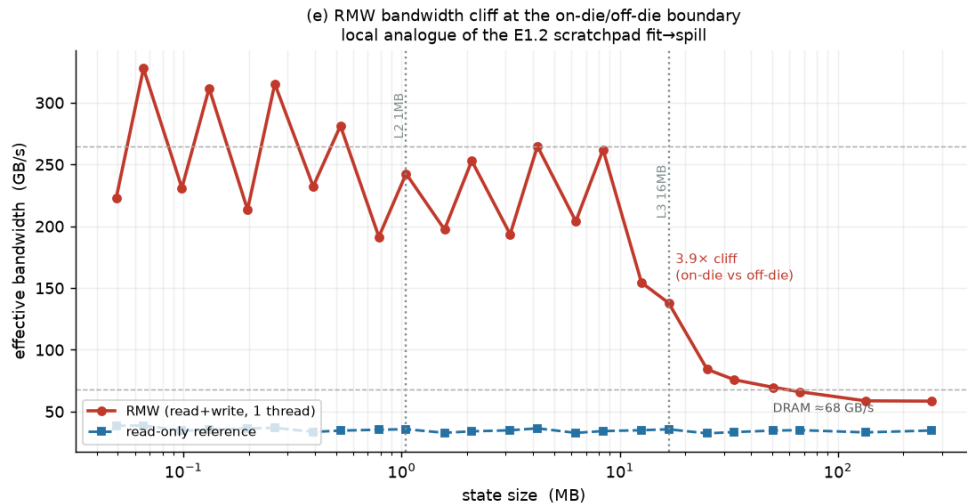
그림 6-1. KV cache vs TTT의 토큰당 트래픽 crossover(자체 실험). 왼쪽: KV는 문맥 S 가 길수록 read 트래픽이 선형 증가하지만, TTT의 full RMW는 문맥과 무관한 상수 — 두 선이 만나는 crossover 지점 S^* 이후로는 고정 state가 이긴다. 오른쪽: crossover 위치가 모델 크기 ($m \cdot d^2$)에 비례해 커진다 — memory가 클수록 TTT가 유리해지는 문맥이 뒤로 밀린다. 출처: 본서 자체 실험 (E1.3 + E3 closed form, <1% 일치). crossover 위치가 주장이다.

직관 고정 크기 state의 이득은 “언제나”가 아니라 **crossover 이후**다. 짧은 문맥에서는 KV cache가 더 싸고, 문맥이 crossover S^* 를 넘어야 고정 state가 이긴다. S^* 는 모델 (memory)이 클수록 뒤로 밀린다. 서빙결정은 이 crossover를 자기 구성에 대입해 계산하는 산수다.

주의 fast-weight state의 숨은 세금 두 가지. (1) $\text{RMW} \approx \text{read의 절반 throughput}$ — KV append는 순수 write라 이 세금을 피하지만, matrix/TTT memory는 매 토큰 큰 state 전체를 읽고 다시 쓴다. (2) state가 on-die(SRAM/L3)에 안 들어가 off-die로 넘어가면 대역폭이 절벽처럼 떨어진다. state를 on-chip에 상주시키려는 이 계열 kernel들의 집착이 여기서 나온다.

표 6-1 — 네 부품의 시스템 요약 (head 당, decode 기준).

부품	state	토큰당 write	decode 병목	KV 대비
KV cache (softmax)	$O(L)$ 증가	append (FLOPs 0)	cache 전체 read	기준선
matrix / linear attn	d^2 고정	rank-1 RMW	state RMW	crossover $L^* \approx d/2$ 후 이득
SSM / Mamba	$d \times N$ 고정	감쇠 + 덧셈 RMW	state RMW	conv state 추가 관리
TTT-Linear	d^2 고정	delta RMW ($\approx 2 \times$ read FLOPs)	state RMW	write에 gradient
TTT-MLP	$\approx 8d^2$	backward 포함 ($\approx 4 \times$ forward)	state RMW + backward	decode에 backward



Source: E2.2 (CPU numpy). SHAPE-only: the 3.9x L3→DRAM cliff structure transports; GB/s values do not (H100 on/off-die ridge ~100x elsewhere). RMW ≈ 0.5x read throughput = the write-back tax KV avoids.

그림 6-2. read-modify-write의 대역폭 절벽 (자체 실험). on-die(L3) 대비 off-die(DRAM)로 넘어가는 경계에서 RMW 대역폭이 약 3.9배 폭 떨어진다. 그리고 RMW(read+write)는 read-only의 약 절반 throughput —이 “write-back 세금”이 바로 KV cache append가 피하고 fast-weight state 가무는 비용이다. 출처: 본서 자체 실험 (E2.2, CPU). shape만 이식되며 (절벽구조), 절대 GB/s는 호스트 의존.

핵심 이 권의 네 부품은 전부 같은 뼈대의 변주다: 고정 크기 state에 매토큰 read-modify-write. 손잡이는 세 개 — (1) 무엇을지울까 (retention gate = 학습된 eviction), (2) 어떻게쓸까 (Hebbian 더하기 → delta 고쳐쓰기 → gradient step), (3) 얼마나 깊은 state(vector → matrix → 작은 MLP). 여섯 논문은 이세 손잡이를 각자 다르게 돌린 것이고, 병렬화(chunkwise)는 이 뼈대를 GPU에올리는 유일한 방법이다.

요약 - **associative memory** = key → value 쓰기 가능한 표. KV cache(무손실·무한 성장)와 matrix memory(고정 크기·lossy)는 같은 추상화의 양 극단이다. 겹쳐 쓰기의 대가는 crosstalk이고, capacity는 parameter 수가 아니라 key 차원($O(d)$)에 묶인다. - **linear attention** = **fast-weight programming**. 누적 행렬에 매 토큰 outer product를 더하는 것. gate(RetNet/GLA)는 오래된 것 지우기, delta(DeltaNet)는 틀린 만큼만 고쳐쓰기, Longhorn은 무조건 안정, RWKV-7은 채널별 벡터 일반화. 전부 한 (state, update, cost) 객체의 손잡이 조합. - **SSM/Mamba** = 새는양동이 + selective gate. step size Δ 가 모든 gate의 기원. Mamba-2의 SSD로 SSM과 linear attention이 한 식으로 합쳐지고, 세 계산경로는 전부 exact(chunk = tiling). - **TTT** = state를 작은 모델의 weight로 승격하고 test-time에 진짜로 학습. TTT-Linear = gate 없는 DeltaNet, surprise = 오차 기반 write. decode에 backward가 들어온다. - **chunkwise** = 블록 안은 병렬·경계만 순차. update가 state-linear면 exact(C = 성능 knob), nonlinear면 anchor 근사(C = **semantic knob**). 품질 최적 C 와 MFU 최적 C 의 이율배반이 TNT의 존재 이유.

다음 권 (G03·Titans)은 이 부품들 위에 **momentum**(최근 놀란 방향의관성)과 **retention gate**와 **deep memory**를 쌓아, TTT의 세 결핍을 메운 첫완성형 아키텍처를 다룬다.